Homs University

IT Faculty /5th year

Knowledge Databases

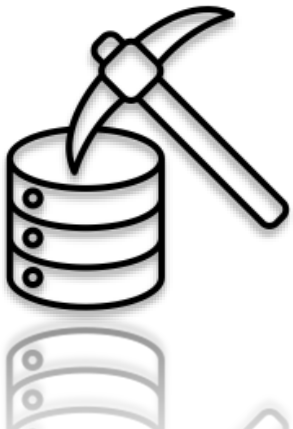
Lab -2-: Getting to know your Data

Introduced By:

Eng. Heba Tadmouri Eng. Khaled Kondakji

Supervised By:

Dr. Eng. Asmaa Shaar



Getting to know your Data

In the era of big data, the ability to quickly analyze data and extract insights is key to making smart decisions. However, raw data is often unstructured, contains errors, or is difficult to understand without processing.

🎯 Lab Goals

Learn how to transform raw data into usable information using:

- **Cleaning** (repairing names, converting types).
- **Statistical Description** (mean, standard deviation, etc.).
- **Visual Representation** (Histogram, Boxplot).

Methodology

1. **Understanding Data Types** (nominal, ordinal, ...)
    - Correcting column names.
    - Converting text to numbers (e.g., price in $).
3. **Quick Statistical Description**:
    - describe() for numeric values.
    - mode() , mean() to compare companies.
4. **Visual Analysis:**
    - Histogram: For mileage distribution.
    - Boxplot: For comparing prices between brands.

Example: Car Sales Data

- **Problem**:
    - Inconsistent data (column names with spaces, prices recorded as text).
    - Difficulty identifying patterns due to lack of clarity.

Attributes:

Attributes has four types:

- Nominal like **Color**, **Marital Status**, ...
- Ordinal like Exam **degrees** (F, D, C, B, A, ...), **Size** (small, medium, large), ...
- Interval-Scaled like **Calendar dates**, **temperature in Celsius**, ...
- Ratio-Scaled like **age**, **height**, **weight**, ... We can also classify attributes to two types (discrete and continuous):

Basic statistical descriptions can be used to give us overall picture of our data, and identify properties of the data and highlight which data values should be treated as noise or outliers.

- **Central Tendency** They include measures like (*mean*, *median*, *mode* and *midrange*).
- **Dispersion** of the data: In this kind we have some measures like (*variance* and *standard deviation*).
- **Graphs.**

Example

we'll use Pandas library to get some statistical descriptions about car sales dataset as an example. The dataset has the following attributes:

#	Attribute	Explanation
1	Make	Manufactured Company.
2	Colour	Car Color.
3	Odometer (KM)	The total distance that car walked.
4	Doors	The number of doors.
5	Price	Car price in $.

I. Import required libraries and load our dataset:

```
import pandas as pd
import plotly.graph_objects as go
import pandas.api.types as pdt
#load car sales dataset:
car_sales_df = pd.read_csv('car sales dataset.csv')
car_sales_df.head()
```

	Make	Colour	Odometer (KM)	Doors	Price
0	Honda	White	35431.0	4	$15,323.00
1	BMW	Blue	192714.0	5	$19,943.00
2	Honda	White	84714.0	4	$28,343.00
3	Toyota	White	154365.0	4	$13,434.00
4	Nissan	Blue	181577.0	3	$14,043.00

How can we quickly and effectively understand car sales data?

The raw data contains:

- Inconsistent names (e.g., "**Colour**" instead of "**Color**").
- Unstructured values (e.g., a **price** written as "$10,000" instead of 10,000).
- Incorrect data types (numbers that read as text).
- Outliers that can distort the analysis.

→ Inaccurate analysis, bad decisions!

Objective:

1. Data cleaning (fixing names, removing spaces, converting types).
2. Statistical description (understanding distribution, central tendency, dispersion).
3. Visualization (quickly detecting patterns and problems).
4. Getting insights (e.g., which car company has the highest prices? Which outliers?).

II. Data Cleaning

1- Cleaning up attribute names:

- **strip()** removes extra spaces from the beginning and end of names
- **rename()** corrects inconsistent names (e.g., **Colour** -> **Color**)

```
car_sales_df.rename(columns={'Colour' : 'Color'}, inplace=True)
car_sales_df.rename(columns=lambda x: x.strip(), inplace=True) #remove whitespaces
car_sales_df.columns
```

```
Index(['Make', 'Color', 'Odometer (KM)', 'Doors', 'Price'], dtype='object')
```

Note: you can reassign `car_sales_df.columns` passing to it a list of new attributes names.

Note: **`strip()`** function removes only leading and trailing spaces **only** from python string.

2. Processing the price attribute:

- Remove the currency symbol ($) using **`str.replace()`**.
- Convert the values to numbers (`float`) to enable calculations.

We use **`replace()`** function to convert the type of price attribute to float:

```
car_sales_df.Price = car_sales_df.Price.str.replace('[\\$\\,\\.]','', regex=
True).astype(float)
car_sales_df.head()
```

	Make	Colour	Odometer (KM)	Doors	Price
0	Honda	White	35431.0	4	$15,323.00
1	BMW	Blue	192714.0	5	$19,943.00
2	Honda	White	84714.0	4	$28,343.00
3	Toyota	White	154365.0	4	$13,434.00
4	Nissan	Blue	181577.0	3	$14,043.00

	Make	Color	Odometer (KM)	Doors	Price
0	Honda	White	35431.0	4	1532300.0
1	BMW	Blue	192714.0	5	1994300.0
2	Honda	White	84714.0	4	2834300.0
3	Toyota	White	154365.0	4	1343400.0
4	Nissan	Blue	181577.0	3	1404300.0

3. Improving data performance:

- Converting text columns with limited values (e.g., **Make**) to category type.
- This reduces memory usage and speeds up data operations. Describe dataset:

- We use **`astype()`** function
- We can treat each attribute which has object type as category type using the following statements:

```
for attr in car_sales_df.columns:
    if pdt.is_object_dtype(car_sales_df[attr]):
        car_sales_df[attr] = car_sales_df[attr].astype('category')
car_sales_df.dtypes
```

```
Make            category
Color           category
Odometer (KM)    float64
Doors              int64
Price            float64
dtype: object
```

Pandas offers a lot of functions to get statistical information about any dataset.

III. Describe dataset:

- `describe()` function: give some statistical information about numeric attributes only such as (mean, std, min, max and more, ….).
  - ✓ We can use `include` parameter to include discrete attributes description.
  - ✓ You can specify an attribute name to get description about it only.

```
car_sales_df.describe(include='all')
#You can use describe function for single attribute like:
#car_sales_df['Odometer (KM)'].describe()
```

	Make	Color	Odometer (KM)	Doors	Price
count	1000	1000	1000.000000	1000.000000	1.000000e+03
unique	4	5	NaN	NaN	NaN
top	Toyota	White	NaN	NaN	NaN
freq	428	440	NaN	NaN	NaN
mean	NaN	NaN	131253.237895	4.011000	1.604281e+06
std	NaN	NaN	67343.562285	0.372851	8.364181e+05
min	NaN	NaN	10148.000000	3.000000	2.796000e+05
25%	NaN	NaN	72503.000000	4.000000	9.798500e+05
50%	NaN	NaN	131253.237900	4.000000	1.496500e+06
75%	NaN	NaN	190259.750000	4.000000	2.032200e+06
max	NaN	NaN	249860.000000	5.000000	5.245800e+06

- `info()` function: gives summary about dataset attributes (total objects, type, and many more).
  - ✓ You can specify an attribute name to get information about it only.

```
car_sales_df.info()
#You can use info function for single attribute like:
#car_sales_df['Odometer (KM)'].info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype   
---  ------          --------------  -----   
 0   Make            1000 non-null   category
 1   Color           1000 non-null   category
 2   Odometer (KM)   1000 non-null   float64 
 3   Doors           1000 non-null   int64   
 4   Price           1000 non-null   float64 
dtypes: category(2), float64(2), int64(1)
memory usage: 25.9 KB
```

Tendency measurements:

- To get central tendency measurements we use built in Pandas functions as follows:

```
print('Odometer attribute mean:',car_sales_df['Odometer (KM)'].mean())
print('Odometer attribute median:',car_sales_df['Odometer (KM)'].median())
```

```
Odometer attribute mean: 131253.237895
Odometer attribute median: 131253.2379
```

- To calculate standard deviation we use `std()` function provided by Pandas library.

```
car_sales_df['Odometer (KM)'].std()# result is 67343.56228522141
```

- To get `Make` attribute mode value we can use `mode()` function as follows:

```
car_sales_df.Make.mode() #result is Toyota
```

Visualization (to discover patterns)

What is Plotly?

Plotly is a powerful and interactive graphing library used in Python for creating highly customizable and interactive plots. Unlike traditional static plots, **Plotly** generates plots that are interactive by default. This means users can zoom in, hover over data points, and click on items to reveal more information.

Key Features of Plotly:

- **Interactive**: Plots are interactive by default, allowing zooming, panning, and tooltips on hover.
- **Wide Variety of Charts**: From basic charts (scatter, line, bar) to complex visualizations (3D charts, choropleth maps, etc.).
- **Easy to Use**: It has a simple syntax for creating visually appealing charts.
- **Web-based**: Plotly graphs can be embedded in web applications, making it great for data visualization on the web.
- **Supports Plotly Express**: A high-level API for rapid visualization with minimal code.

How Plotly Works:

1. **Creating a Figure**:
   - `fig = go.Figure()` creates a blank figure object.
2. **Adding Traces**:
   - `fig.add_trace(go.Scatter(...))` adds a trace (a specific chart or graph element) to the figure.
3. **Displaying the Plot**:
   - `fig.show()` renders the plot.

Use Cases for Plotly**:**

- **Data Analysis**: Visualize data trends, distributions, and correlations.
- **Dashboards**: Build interactive web-based dashboards for analytics.
- **Reports**: Embed interactive plots in web applications or Jupyter notebooks.
- **Data Science Projects**: Create beautiful and interactive charts with minimal code.

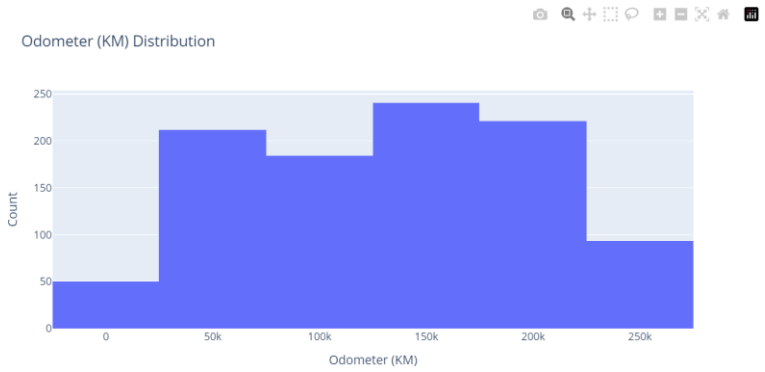
Plotly vs. Matplotlib Comparison:

Feature	Plotly	Matplotlib
Interactivity	Interactive by default	Static by default (can add interactivity)
Ease of Use	Simple, especially with Plotly Express	More verbose, requires more code
Customization	Easy and quick customization	Highly customizable but more complex
Performance	Good for large datasets (interactive)	Best for static plots with smaller datasets
Output	Interactive plots (web-friendly)	Static plots (can be embedded in web)
Web Integration	Easy (via Dash, web embedding)	Requires more work for web integration

1. Histogram

- To understand the distribution of data (frequencies).
- To view the distribution of `Odometer (KM)` attribute, we can use **`Histogram`** Class, by discretizing it into 5 separate bins and counting the frequency for each bin using `bins` parameters.

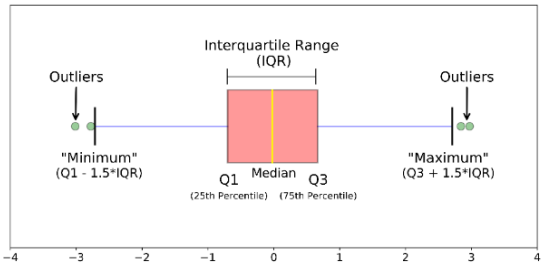
```
# Create Figure Object:
fig = go.Figure()
# Adding a trace which contains Histogram object
fig.add_trace(go.Histogram(x=car_sales_df['Odometer (KM)'], nbinsx=5))
# Update Axis and Title:
fig.update_layout(
    title='Odometer (KM) Distribution',
    xaxis_title='Odometer (KM)',
    yaxis_title='Count')
```



2. Boxplot

Purpose: To compare distributions between categories (such as brands).

Boxplot displays five-number summary of a distribution minimum, Q1, Median, Q3, maximum.



- **Box** class from `plotly` library is used for adding a trace for each brand by extract its Price values using the following code:

```
# Create Figure Object:
box_fig = go.Figure()
# extract Make attribute unique values:
makes = car_sales_df.Make.unique()
# Adding Boxplot Traces:
# name parameter for legend
for v in makes:
        box_fig.add_trace(go.Box(y=car_sales_df[car_sales_df['Make']==v].loc[:,"Price"]
, name=str(v)))
# Update Axis and Title:
box_fig.update_layout(
    title='Companies Sales Five-Number Summary',
    yaxis_title='Price')
```

To understand how boxplot is worked let's look at the following example:

- For Toyota, we see that median price of items sold is 15000, Q1 is 10000, Q3 is 20000 and we have some outlying observations were plotted individually, as their values are more than Q3 + 1.5*IQR.